1. General Information

The protocol is based on requests sent trough POST method to the web service running on the Automatic Cash.
In the following paragraphs, it will be detailed the structure of the requests accepted by the web service.
The structure is based on the JSON data-exchange format.

In this first draft we will not take into account the communication security.
Later, if needed, we will discuss about data encryption (for example using https and a login procedure).

2. Webservice

The web service URL that is listening for POST requests is the following:

https://IP_MACCHINA/selfcashapi/

where IP_MACHINE is the local IP address of Automatic Cash (configurable through the proper configuration menu).
If the client does not receive the web service answer in 5 seconds, it has to send again the request.

3. List of supported commands

The commands supported by Automatic Cash are the following:

1) Payment request: type = 1
2) Polling to check the status of the payment: type = 2
3) Delete pending payment: type = 3
4) List of pending payments: type = 5
5) Operator withdrawal request: type = 10
6) Polling to check the status of the operator withdrawal: type = 11
7) End of operator withdrawal: type = 12
8) Machine status: type = 20
9) Start refill: type = 30
10) End refill: type = 31
11) Manual refill: type = 32
12) Polling refill: type = 33
13) Operator login: type = 40
14) Operator logout: type = 41
15) Hopper emptying: type = 50
16) Recycler emptying: type = 51
17) Stacker cancellation: type = 52
18) Emptying polling: type = 53
19) Coin dispenser emptying: type = 54
20) Door opening: type = 55
21) Cash closing: type = 60
22) Refund: type = 65
23) Polling Refund: type = 66
24) Withdrawals list: type = 70
25) Payments list: type = 71
26) Openings list: type = 72
27) Cash closing list: type = 73
28) Access configuration menu: type = 80
29) Reboot/Shutdown: type = 81
30) Machine Version: type = 82
31) Peripherals configuration: type = 90
32) Software configuration: type = 91
33) Petty cash setting: type = 92
34) End petty cash restore: type = 93

1) Payment request

The Automatic Cash will manage the payment of the specific amount passed as a parameter.
The parameters are the following:

{ "tipo": 1, "importo": 590, "opName": USERNAME, "operatore": ID_OPERATORE.
"refundable": 0/1, "credit_card": 0/1, "authKey": "12345678" }

The amount must be written in cents; in the example above, the amount is 5,90 €.

The field "**opName**" is optional and identifies the operator that do the request to the system;
This field if present does automatically the login when the payment is processed by the Automatic Cash

Warning: When the "opName" is present, it's not necessary to do the login through the touch screen or with the "Operator Login" request.
Otherwise the without the "opName" field, the login must be done through the touch screen or the "Operator Login" request, before sending the payment request, like explained in paragraph 11)

The field "**operatore**" is optional, in case the client system uses an operator identifier to identify the terminal that sent the request.
Warning: This field is deprecated from version 2.0 of the protocol, is present only for backward compatibility.

The field "**refundable**" is optional:
- if not present or set to 1, the payment can be refunded with the apposite procedure
- if set to 0, the payment cannot be refunded

The field "**credit_card**" is optional:
- if not present or set to 0, the payment will be performed in cash
- if set to 1, the payment will be performed, for the models that support this functionality, with credit/debit card.

Warning: Tag "**authKey**" contains the authentication key and it's optional and needed **ONLY FOR SPECIFIC INSTALLATIONS**.
If the tag is required but not present or not valid, server response will be error code 120.
Before proceeding with the implementation of this specific functionality, please contact VNE for further info.

The Automatic Cash response is like the following, where is present the type of request, the status of the request, the value of payment requested and the unique identifier.

{ “tipo”: 1, “importo”: 590, “req_status”: 1, “id”: “1454496777001” }

The “**req_status**” field value is shown in Table 1:

req_status: 1	Request received and managed correctly (ACK)
req_status: 2	Failed request (NACK)

Table 1: req_status

If “**req_status**”: 2 there has been a problem and the machine cannot accept the request.
In this case there will be an additional parameter “mess” that will contain a code explaining the NACK reason. All the possible NACK messages and the solutions available are detailed in **Appendix I.**

The transaction unique identifier is made up by adding a 3-digit incremental number to the UNIX timestamp expressed in seconds.
This identifier has to be used in the following polling request to check the status of the payment.

2) Polling to check the status of the payment

With this request, you ask to Automatic Cash info about the status of a payment previously sent.
The parameters are the following:

{ "tipo": 2, id: "1454496777001", "opName": USERNAME }

where "**tipo**" is the request type, "**id**" is the unique identifier of the transaction returned as a response by a previous Payment request.
The field "**opName**" is optional and identifies the operator that has sent the request.

The Automatic Cash response reports the type of the request ("**tipo**" field), the status of the request
("**req_status**" field, the unique identifier of the transaction ("**id**" field), and the current status of the payment ("**payment_status**" and "**payment_details**" fields).

Example: payment completed response:

{ "tipo": 2, "req_status": 1, "id": "1454496777001", "payment_status": 1
"payment_details": { "amount": 590, "inserted": 1000, "rest": 410 },
}

Example: payment in progess response:

{ "tipo": 2, "req_status": 1, "id": "1454496777001", "payment_status": 2
"payment_details": { "amount": 590, "inserted": 200, "rest": 0, "status": "pending", "not_returned": 0 }
}

The "**req_status**" values are detailed in Table 1 in paragraph 1.

The **payment_status** field details the current status of the payment.
These are the two possible values:

payment_status: 1	Payment completed
payment_status: 2	Payment still in progress

Tabella 2: payment_status

If "payment_status": 2 the client has to wait a brief time (for example 1 second) and send again the request, until it will receive a response with "**payment_status**": 1.

The "payment_details" field contains the details of the payment:

"payment_details": { "amount": 590, "inserted": 1000, "rest": 410,
"status": completed/returned/partial/pending, "not_returned": 0 }

The "**payment_details**" subfields are the following:

- "**amount**": payment amount
- "**inserted**": amount inserted into Automatic Cash by the customer
- "**rest**": change automatically dispensed by Automatic Cash
- "**status**": payment status:

  - **completed**: payment successfully completed
  - **returned**: payment canceled, and the amount inserted by the customer (if present) has been returned to the customer.
  - **partial**: payment canceled, a partial payment has been accepted.
  It means that the amount inserted by the customer was lower than the due amount, but the operator has chosen to set the payment as completed.
  - **deleted**: payment has been deleted before the insertion of money by the customer
  - **pending**: payment still in progress
  - **notCompleted**: payment cannot be completed, because Automatic Cash cannot complete the dispensing of the change amount.
  The amount not returned can be calculated using the following formula:
  not_paid_amount = inserted – (amount + rest)

  In this case there are two possible solutions:
    1) perform a refill of Automatic Cash, to insert the banknotes/coins needed to complete the change dispensing.
    After refill operation, Automatic Cash will try automatically to dispense the remaining change
    2) clear the current payment by selecting the option to accept a partial payment

- "**not_returned**": this subfield is used only when status is *pending* following a payment cancellation with return of the amount inserted. If the dispensing of the amount cannot be completed (e.g. the machine has not enough cash), flag not_returned has value 1. In all other cases it has value **0**.

Warning: if it has been selected a payment using credit/debit card, once it has been completed the **field "inserted" will take value -2**.

Warning: if the payment is in a canceled stutus ("returned", "partial" or "deleted") there is the tag **"cancelled_by"** with the user that had requested the cancellation.

The operator can also cancel the payment using the proper menu button.
In this case "payment_details" contains the amount inserted by the customer until the time of the cancellation and the "**status**" field will have the value "**returned**" or "**partial**" or "**deleted**", according to the cancellation type.
If the payment has been deleted before the money insertion, al the "**payment_details**" fields will have value 0.

Example:
"payment_details": { "amount": 590, "inserted": 200, "rest": 200, "status": "returned", "not_returned": 0, "cancelled_by": "op1"}

In the example above the payment has been canceled by user "op1" after the customer has inserted 2 Euro and the Automatic Cash has returned the 2 Euro to the customer.

3) **Delete pending payment:**

With this request Automatic Cash cancels a still pending payment.

Parameters are the following:

{ "tipo": 3, id: "1454496777001", "tipo_annullamento":1/2, "opName": USERNAME }

"**id**" is the unique identifier of the transaction returned as a response by a previous Payment request.

The field "**opName**" is optional and identifies the operator that has sent the request.
If present, the operation will be saved with the specified operator, otherwise will be saved with the actual logged in operator.

Warning: field "**opName**" is mandatory **ONLY FOR SPECIFIC INSTALLATIONS**.
For these implementations, the field has to have the following values so that the cancellation will be successful:

- same value of field "opName" in the Payment request (that is the user who cancels the payment must be the same user who requested it)
- opName = "", that is empty field; in this case the user is a "super_user" who has the rights to cancel every payment.

If these conditions are not satisfied, it will be returned the error code **114** and the payment will not be canceled.
Before proceeding with the implementation of this specific functionality, please contact VNE for further info.

The "**tipo_annullamento**" field specifies the cancellation mode; it can have the value 1 or 2.
We can have four different types of pending payment cancellation:

1) payment cancellation before it has been processed by Automatic Cash.
In this case the value of the "**tipo_annullamento**" field has no meaning and the payment will be deleted immediately

2) cancellation of a payment that has already been started but the customer has yet to insert the requested amount.
In this case the value of the "**tipo_annullamento**" field causes the following results:

value = 1: accept partial payment.
The pending payment is canceled and the amount already inserted by the customer is maintained and saved in database.

value = 2: return.
The pending payment is canceled and the amount already inserted is returned to the customer.

3) cancellation of a payment, whose change dispensing operation has already been started by Automatic Cash. The customer has inserted an amount higher than the amount requested, but Automatic Cash cannot dispense the exact change due.

valore = 1: accept partial payment. Pending payment is canceled and set as partially accepted.

valore = 2: return. Pending payment is returned only if the following condition occurs:

- amount inserted by the customer is greater than the amount due, but the machine HAS NOT paid ANY change.

  Attenzione: if the machine has already dispensed a partial rest, a return will no longer be possible.

Automatic Cash response contains the type of the request ("tipo" field), the request status ("req_status" field), and the payment status at the moment of the cancellation ("payment_status" and field).

The **"req_status"** field value is detailed in Table 1 in paragraph 1.
Please take not of the NACK response with mess=104.
This case can occur in two different situations:

→ "id" parameter does not match an existing "id"

→ "id" parameter matches an "id" of a payment that has already been deleted with a "payment_status: 1" response.

Warning: this payment has been deleted before being processed by the machine, so it will not appear in any transaction/payment list.

The **"payment_status"** field can take the following values:

payment_status: 1	Payment deleted; payment has been deleted before the Automatic Cash has processed it and put it on screen
payment_status: 2	Payment in progress; the cancellation will be performed as soon as possible. Check the operation result using the polling request.
payment_status: 3	Cannot cancel payment: payment already completed or deleted
payment_status: 4	Cannot return amount. To cancel the payment choose the "**accept partial payment**" mode

Warning: after receiving a payment_status = 2, client has to continue to send the polling request to check the status of the operation.
In this case the payment operation has already been started by the machine: the cancellation will be performed as soon as possible, but there cannot be any guarantee on the timing.

Warning: in case of cancellation with return of the cash, there's the worst-case where the machine cannot return the full amount inserted.
In this case the following polling will report ***"status": "pending"*** and ***"not_returned" : 1*** to point out that the return has not been completed.
Now it's possible to choose if deleting the payment by sending again the "Delete pending payment" command or do a refill to ensure the machine has enough cash to complete the return.

4) List of pending payments

Automatic Cash will return the list of all the payment currently pending.

Parameters are the following:

{ “tipo”: 5, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that has sent the request.

Automatic Cash response contains the list of all the unique identifiers of the pending payments and the length of this list.

If there is one or more pending payment, the response contains the field “**list_length**” whose value is higher than 0, and the “**pending_list**” field contains the list of these payments.

{ “tipo”: 5, “list_length”: 2,
“pending_list”: {“id_0”: 1454496777001, “id_1”: 1454496777002 }
}

Payment list is made up by the “**id_N**” field and the correspondent id code, where N represent the payment position into the list: index 0 will be the oldest pending payment (the one currently on the customer screen).

If there are no pending payments, response will only contain the “**list_length**” with value 0.

{ “tipo”: 5, “list_length”: 0 }

5) **Operator withdrawal request**

Using this request, Automatic Cash allows the operator to withdraw from the machine a specific amount passed as parameter.

Parameters are the following:

{ "tipo": 10, "importo": 5000,
"opName": USERNAME, "operatore": ID_OPERATORE,
"taglio": "monete"/"banconota_2000" }

The field "**importo**" is the amount to withdraw and must be written in cents; in the example above, the amount is 50,00 €.

The field "**opName**" is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the withdrawal is processed by the Automatic Cash

The field "**operatore**" is optional, in case the client system uses an operator identifier to identify the terminal that sent the request.
Warning: This field is deprecated from version 2.0 of the protocol, is present only for backward compatibility.

The "**taglio**" field is optional: if present, it forces the start of the dispensing operation without the need that the operator uses the proper interface.
The operator login is performed automatically and the machine immediately starts to dispense the requested amount.

The field can take four different types of value:

"taglio": "monete" - withdrawal amount will be dispensed in mixed coins
"taglio": "monete_XXX" - withdrawal amount will be dispensed with the coin denomination passed as a parameter. The value of the coin has to written in cents.
"taglio": "banconota_XXXX – withdrawal amount will be dispensed with the banknote denomination passed as a parameter. The value of the banknote has to written in cents.
"taglio": "all" - withdrawal amount will be dispensed using both banknotes and coins. Payment starts with the higher denominations of notes and finishes, if necessary, with coins.

Example: "taglio": "banconota_2000" means the withdrawal amount will be paid out, if possible, with 20 Euro banknotes.
Example: "taglio": "monete_100" means the withdrawal amount will be paid out, if possible, with 1 Euro coins.

Automatic Cash response contains the request type, the request result, the withdrawal amount and a unique identifier of the transaction.

{ "tipo": 10, "importo": 5000, "req_status": 1, "id": "1457596777018" }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

If "**req_status**": 2 there has been a problem and the machine cannot accept the request.

In this case there will be an additional parameter "**mess**" that will contain a code explaining the NACK reason. All the possible NACK messages and the solutions available are detailed in **Appendix I**.

Warning: Automatic Cash can answer with a NACK response with mess=106: it means that the chosen denomination is not available (for example, the machine does not contain enough 20 Euro banknotes to perform the withdraw of the given amount).

The transaction unique identifier is made up by adding a 3-digit incremental number to the UNIX timestamp expressed in seconds.
This identifier has to be used in the following polling request to check the status of the withdraw.

6) Polling to check the status of the operator withdrawal

With this request, you ask to Automatic Cash info about the status of a withdraw previously requested.
The parameters are the following:

{ "tipo": 11, id: "1457596777018", "opName": USERNAME }

where "**tipo**" is the request **type,** "**id**" is the **unique identifier** of the transaction returned as a response by a previous Operator withdrawal request.

The field "**opName**" is optional and identifies the operator that do the request to the system;

Example in case of withdrawal completed:

{ "tipo": 11, "req_status": 1, "id": "1457596777018", "withdraw_status": 1
"payment_details": { "amount": 5000, "withdrawed": 5000 }
}

Example in case of withdrawal still pending:

{ "tipo": 11, "req_status": 1, "id": "1457596777018", "withdraw_status": 2 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

The field "**withdraw_status**" contains the current status of the withdrawal:

withdraw_status: 1	Withdrawal completed
withdraw_status: 2	Withdrawal not completed

Tabella 3: withdraw_status

If "withdraw_status": 2, client has to wait a brief time (for example 1 second) and send again the request, until it receives a response with "withdraw_status": 1.

If the withdrawal has been completed ("withdraw_status": 1) there is an additional field "**payment_details**" containing the details of the withdrawal:

“payment_details”: { “amount”: 5000, “withdrawed”: 5000 }

The subfields of “payment_details” are the following:

- “**amount**”: starting withdrawal amount
- “**withdrawed**”: amount actually withdrawed by the operator (in case of partial withdrawal this value will be lower than “**amount**”)

7) End of operator withdrawal

With this request Automatic Cash leaves the withdrawal mode and comes back to the payment mode.

The parameters are the following:

{ “tipo”: 12, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that do the request to the system;

Automatic Cash response only contains the result of the call.

{ “tipo”: 12, “req_status”: 1 }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.
If “**req_status**”: 2 there has been a problem and the machine cannot accept the request.
In this case there will be an additional parameter “**mess**” that will contain a code explaining the NACK reason. All the possible NACK messages and the solutions available are detailed in **Appendix I**.

Please take note to send this command only after having sent the polling to check the result of the withdrawal and having received a **“withdraw_status”: 1** response (that means the withdrawal has been completed).

8) **Machine status**

Automatic Cash will return info about the status and the content of its peripherals: the banknotes recycler (called '**recycler**') and the coin dispenser (called '**hopper**').

The parameters are the following:

{ “tipo”: 20, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that do the request to the system;

Automatic Cash response reports the request type and a JSON dictionary containing status and content of each peripheral.

Example:

{ “tipo”: 20,
“recycler”: { “error”: 0, “mess”: “OK”,
“banconota_1”: {“valore”: 500, “quantita”: 25, “alert”: 0/1/2,
“min_level”: 20, “max_level”: 100 },
“banconota_2”: {“valore”: 1000, “quantita”: 20, “alert”: 0/1/2,
“min_level”: 30, “max_level”: 80 },
“stacker”: 12000,
“totalInRecycle”: 32500,
“detailNotesStacker”:{ 500: 0, 1000: 10, 2000: 1, 5000: 0, 10000:0, 20000:0,50000:0}},

“hopper”: { “error”: 0, “mess”: “OK”,
“moneta_1”: 121,
“moneta_1_alert”: 0,
“moneta_1_min_level”: 80, “moneta_1_max_level”: 500,
“moneta_1_accept”:1, “moneta_1_recycle”: 1, “moneta_1_cashbox”:0,
“moneta_2”: 55,
“moneta_2_alert”: 0,
“moneta_2_min_level”: 80, “moneta_2_max_level”: 500,
“moneta_2_accept”:1, “moneta_2_recycle”: 1, “moneta_2_cashbox”:0,
“moneta_5”: 107,
“moneta_5_alert”: 0,
“moneta_5_min_level”: 80, “moneta_5_max_level”: 500,
“moneta_5_accept”:1, “moneta_5_recycle”: 1, “moneta_5_cashbox”:0,
“moneta_10”: 23,
“moneta_10_alert”: 0,
“moneta_10_min_level”: 80, “moneta_10_max_level”: 500,
“moneta_10_accept”:1, “moneta_10_recycle”: 1, “moneta_10_cashbox”:0,
“moneta_20”: 37,
“moneta_20_alert”: 0,
“moneta_20_min_level”: 80, “moneta_20_max_level”: 500,
“moneta_20_accept”:1, “moneta_20_recycle”: 1, “moneta_20_cashbox”:0,
“moneta_50”: 56,
“moneta_50_alert”: 0,
“moneta_50_min_level”: 80, “moneta_50_max_level”: 500,
“moneta_50_accept”:1, “moneta_50_recycle”: 1, “moneta_50_cashbox”:0,

“moneta_100”: 112,
“moneta_100_alert”: 0,
“moneta_100_min_level”: 80, “moneta_100_max_level”: 500,
“moneta_100_accept”:1, “moneta_100_recycle”: 1, “moneta_100_cashbox”:0,
“moneta_200”: 167,
“moneta_200_alert”: 0,
“moneta_200_min_level”: 80, “moneta_200_max_level”: 500,
“moneta_200_accept”:1, “moneta_200_recycle”: 1, “moneta_200_cashbox”:0,
“totalInRecycle”: 49136,
“stacker”: 72
}
}

Attention: if it is used a reycler with **more than two denomination in recycle**, it will be present additional fields name **“banconota_X”**.
For example, if there are four denomination in recycle, you will have the additional subfields **“banconota_3”** and **“banconota_4”**.

The subfields for the ‘**recycler’** peripheral are the following:

- “**error**”: 0 if there is no error, 1 in case of error
- “**mess**”: “OK” if there is no error; in case of peripheral error there will be the error code (in the next protocol version there will be a table with all the possible error codes)
- “**banconota_1**”: first recycler cassette; subfield “**valore**” is the value expressed in cents of the cassette (5€ in the example), “**quantita**” is the number of banknotes contained (25 in the example);
“**alert**” indicates the level of the recycler cassette: 0 if it’s **ok**, 1 if it’s **low** (lower than 6 pieces), 2 if it’s completely **empty;**
“min_level” is the petty cash level (20 pieces in the example);
“max_level” is the maximum number of recyclable notes in the cassette (100 in the example)
- “**banconota_2**”: first recycler cassette; subfield “**valore**” is the value expressed in cents of the cassette (10€ in the example), “**quantita**” is the number of banknotes contained (20 in the example);
“**alert**” indicates the level of the recycler cassette: 0 if it’s **ok**, 1 if it’s **low** (lower than 6 pieces), 2 if it’s completely **empty.**
“min_level” is the petty cash level (30 pieces in the example);
“max_level” is the maximum number of recyclable notes in the cassette (80 in the example)

[possible tags banconota_3, banconota_4 etc…]

- “**stacker**”: recycler stacker content expressed in cents (120 Euro).

- “**totalInRecycle**”: total content of the recycler (excluding the stacker) expressed in cents

- “**detailNotesStacker** ”: dictionary that contains the stacker content detail. The dictionary is made of couple key-value, where key is the denomination of the banknotes and value is the number of pieces in the stacker for that denomination. In the example above, there are ten 10€ notes and one 20€ note in the stacker,

The subfields for the '**hopper'** peripheral are the following:

- "**error**": 0 if there is no error, 1 in case of error
- "**mess**": "OK" if there is no error; in case of peripheral error there will be the error code (in the next protocol version there will be a table with all the possible error codes)
- "**moneta_1**": number of 1 cent coins contained in the hopper (121 pieces in the example)
- "**moneta_2**": number of 2 cents coins contained in the hopper
- "**moneta_5**": number of 5 cents coins contained in the hopper
- "**moneta_10**": number of 10 cents coins contained in the hopper
- "**moneta_20**": number of 20 cents coins contained in the hopper
- "**moneta_50**": number of 50 cents coins contained in the hopper
- "**moneta_100**": number of 1 Euro coins contained in the hopper
- "**moneta_200**": number of 2 Euro coins contained in the hopper
- "**moneta_1_alert**": indicates the level of the denomination in the hopper:
    - **0** if it's ok
    - **1** if it's low (lower than 20 pieces)
    - **2** if this denomination cannot be paid out (0 pieces or level too low)
    - **3** if this denomination is not recycled (not configured to be used for payments)
- "**moneta_1_min_level**": indicates the petty cash level for 1 cent coin (80 pieces in the example)
- "**moneta_1_max_level**": indicates the maximum level for 1 cent coin (500 pieces in the ex.)
- **"moneta_1_accept"**:
    - **1** indicates the denomination of coin is accepted
    - **0** indicates the denomination is not accepted, and will be refused during counting
- **"moneta_1_recycle"**:
    - **1** indicates the denomination of coin is not used for the payment
    - **0** indicates the denomination of coin will not be used for the payment; when the coin is found, the hopper automatically put it in cashbox
- "**moneta_1_cashbox**": indicates the number of 1 cent coins in the coins cashbox (0 pieces in the example)

[tags alert, min_level, max_level for all coin denominations supported]

- "**stacker**": coins cashbox content expressed in cents (0.72 Euro in the example)

- "**totalInRecycle**": total content of the Hopper (excluding the stacker) expressed in cents

In the supported machine models, there will also be a 3 denomination banknote dispenser (called 'dispenser').
The structure of the 'dispenser' field is as follows:

“dispenser”: { “error”: 0, “mess”: “OK”,
“banconota_1”: {“valore”: 500, “quantita”: 250, “alert”: 0/1/2, “error”: 0/1},
“banconota_2”: {“valore”: 1000, “quantita”: 200, “alert”: 0/1/2, “error”: 0/1},
“banconota_3”: {“valore”: 2000, “quantita”: 200, “alert”: 0/1/2, “error”: 0/1},
“totalInRecycle”: 725000,
“rejectBin”: 2
}

The subfields for dispensers are as follows:

- “**error**”: 0 if there is no error, 1 in case of error
- “**mess**”: “OK” if there is no error; in case of peripheral error there will be the error code

- “**banconota_1**”: first dispenser cassette of the dispenser:
  “**valore**” is the centesimal value of the cassette (€ 5 in the example);
  “**quantita**” is the number of notes in the cassette(250 in the example);
  “**alert**” indicates the level of the drawer: 0 if it is ok, 1 if it is low (less than 20 pieces), 2 if it is empty;
  “**error**” indicates whether that particular cassette is in an error state (1) or is ok (0).

The same structure is repeated for dispensing cassette 2 and 3.

- “**totalInRecycle**”: total content in the dispenser expressed in cents

- “**rejectBin**”: number of banknotes contained in the dispenser waste drawer (2 in the example)

In the supported machine models, there will also be 2 hoppers for paying out single denomination coins (structure called ‘**coinDisp**’).
The structure of the ‘**coinDisp**’ field is as follows, with two subfields ‘**hopper**’:

“coinDisp”: {
“hopper_1”: {“valore”: 200, “quantita”: 250, “alert”: 0/1/2, “error”: 0/1},
“hopper_2”: {“valore”: 100, “quantita”: 200, “alert”: 0/1/2, “error”: 0/1}
}

The subfields for **hopper** are:

- “**valore**”: centesimal value of the coin denomination (€ 2 in the example for hopper_1)
- “**quantita**”: number of coins (250 in the example);
- “**alert**”: indicates the level: 0 if it is ok, 1 if it is low (less than 20 pieces), 2 if it is empty;
- “**error**”: indicates whether thae hopper is in an error state (1) or is ok (0).

9) **Start refill**

With the start refill command, Automatic Cash automatically enters refill mode; in this mode the recycler and the hopper are able to accept banknotes and coins.

The parameters are the following:

{ "tipo": 30, "opName": USERNAME, "acceptAll": 0/1 }

The field "**opName**" is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the refill is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

The field "**acceptAll**" is optional and tells the machine to accept also the banknotes that aren't recyclable.

The values can be the following:

1. **acceptAll** = 0 only recyclable banknotes will be be accepted
2. **acceptAll** = 1 will be accepted all banknotes

Automatic Cash response contains the request result.

{ "tipo": 30, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

Warning: do not send the Start refill command while Automatic Cash is performing any other operation.

Warning: the field **acceptAll = 1** is used for the "change" functionality.
In case it is needed to perform a cash change, the suggested procedure is the following:

1. Start refill with acceptAll = 1
2. Insert cash (all banknotes denominations will be accepted)
3. End refill
4. Get the inserted value during refill by calling "**Polling refill**" ("tipo"=33)
5. Send a withdrawal with an amount equal to the value just inserted

10) End refill

With the end refill command, Automatic Cash automatically exits the refill mode and comes back to the payment mode.

The parameters are the following:

{ “tipo”: 31, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that do the request to the system.

Automatic Cash response contains the request result.

{ “tipo”: 31, “req_status”: 1 }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

Warning: if a refill with acceptAll = 0 (see previous paragraph) has been made, once the machine receives the end refill command it will automatically stir the coins inside the hopper for about 30 seconds. Before doing other operations, please wait that the stir operation is completed.

Vice versa, in case of a refill with acceptAll = 1 (“change” refill) the stir is not executed, so it’s possible to immediately launch a new operation.

11) Manual refill

With the manual refill command, the amount of coins and banknotes inside the Automatic Cash can be set.

Warning: be very careful when using this functionality, because if the values inserted are wrong (that is they are different from the real contents of the machine) there will be problems and errors during the dispensing operations.

The parameters are the following:

{ "tipo": 32, "opName": USERNAME, "refill": REFILL }

The field "**opName**" is optional and identifies the operator that do the request to the system;
This field if present does automatically the login when the refill is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

The "**refill**" tag is a dictionary containing the refill values we want to set.

- for the recycler cassettes manual refill, the tags used are **"cassette_refill_1"** and **"cassette_refill_2"** followed by the number of banknotes currently inside the cassettes.
  Warning: not all the recycler models support the manual refill functionality, please check the technical manual for further information.
  If a manual refill for a recycler that doesn't support this function is sent, no changes will occur in the peripheral counters.

- for Smart Hopper manual refill, for each denomination supported by the hopper we use the tag **"hopper_refill_XX"**, where **XX** is the value of the requested denomination expressed in cents, followed by the number of coins of that denomination currently inside the hopper.
  Ex: **hopper_refill_50** refers to 50 cents, **hopper_refill_100** to 1€

- for the refill of the Fujitsu Dispenser, in the models where supported, for each of the 3 dispensing cassettes the tags "disp_refill_1", "disp_refill_2" and "disp_refill_3" are used followed by the number of banknotes currently present in the cassettes.

Example:

"refill":
{
"cassette_refill_1": 27, # recycler cassette 1 contains 27 banknotes
"cassette_refill_2": 10, # recycler cassette 2 contains 10 banknotes
"hopper_refill_10": 110, # hopper contains 110 coins of 10 cents
"hopper_refill_20": 25, # hopper contains 25 coins of 20 cents
"**hopper_refill_100**": **90**, # hopper contains 90 coins of 1€
"hopper_refill_200": 50, # hopper contains 50 coins of 2€
}

If the banknote dispenser is also present, simply add the following tags:

"disp_refill_1": 250, # cassette 1 of the dispenser holds 250 banknotes
"disp_refill_2": 200, # cassette 2 of the dispenser holds 200 banknotes
"**disp_refill_3**": **300**, # cassette 3 of the dispenser holds 300 banknotes

If the single denomination hoppers (coin dispensers) are also present, simply add the following tags:

"coinDisp_refill_1": 250, # hopper 1 holds 250 monete
"coinDisp_refill_2": 200, # hopper 2 holds 200 monete

Warning: no change will be applied for the recycler cassettes and the coins denominations **not presents** in the "refill" dictionary.

Warning: the value associated to each tag represents the **number of pieces currently inside the machine**.
If the machine contains 70 coins of 1€ and other 20 are added, the value to use in the manual refill command will be "**hopper_refill_100**"**: 90**.

Warning: if the value inserted for a single field is NOT allowed (for example a recycler cassette set to refill value 200), no change related to that field will be performed on the machine.

Automatic Cash response just confirms the machine has received the manual refill request:

{ "tipo": 32, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

To be sure that the operation has been successfully completed, it is necessary to check the machine current status through the proper command Status.

12) Polling refill

With the polling refill command, it is possible to poll in real time the amount of money entered during an automatic refill operation.

The parameters are the following:

{ “tipo”: 33 }

Automatic Cash response contains the request result.

{ “tipo”: 33, “req_status”: 1, “refillOn”: 1, “amountRefill”: 2200}

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

The field “**refillOn**” indicates if automatic refill operation is in progress:

“refillOn” = 1 indicates that an automatic refill operation is in progress
“refillOn” = 0 indicates that no refill operations are in progress

The "**amountRefill**" field shows the total expressed in cents of the money inserted in the current automatic refill operation.

Warning: the “**amountRefill**” value is the sum of the total of both coins and banknotes inserted.

In the example above, an automatic refill operation is in progress at the time of polling and € 22 has been introduced so far.

After the "**End Refill**" command has been sent, the subsequent polling refill will receive the following message as a response:

{ “tipo”: 33, “req_status”: 1, “refillOn”: 0, “amountRefill”: 5500}

which indicates that the refill has been completed and the total entered was 55 €.

13) Operator login

Automatic Cash automatically perform the operator login and enters in payment mode.
The basic user for this type of login is called "**auth_login**".

The parameters are the following:

{ "tipo": 40, "opName": USERNAME }

"opName" field is optional: if not present, login will be made as the standard user "**auth_login**". If present, login will be made as the username passed as parameter.

Attention: it is not necessary that the "opName" is equal to a user actually registered on the Automatic Cash.

Attention: all operations performed on Automatic Cash are saved as performed by the operator logged in at the time.
If there is no operator logged in, operations will be saved using the standard username "auth_login".
Thus it's strongly suggested to always send a Login with a specific username before performing any operation.

Automatic Cash response contains the request result.

{ "tipo": 40, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

14) Operator logout

Automatic Cash automatically perform the operator logout and exits from payment mode.

The parameters are the following:

{ “tipo”: 41, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that do the request to the system.

Automatic Cash response contains the request result.

{ “tipo”: 41, “req_status”: 1 }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

Warning: logout cannot be performed if there is a pending payment.
In this case Automatic Cash answers with a 107 error code.

15) Hopper emptying

Automatic Cash starts the hopper emptying routine.

The parameters are the following:

{ “tipo”: 50, “opName”: USERNAME, “full”: 0/1 }

The "**full**" field is optional and indicates whether the emptying of the hopper must be total or partial (emptying to petty cash level).
The possible values are as follows:

1. full = 0 Total emptying
2. full = 1 emptying to petty cash level

If the field is omitted, the default value is 0-total emptying.

The field “**opName**” is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the emptying is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the “auth_login”

Automatic Cash response only confirms the start of the emptying operation.

{ “tipo”: 50, “req_status”: 1 }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

Warning: do not send the Hopper complete emptying command while Automatic Cash is performing any other operation.

Warning: do not send other commands to Automatic Cash until the hopper emptying operation has been completed.
To check when the emptying is completed use the command Emptying polling (see paragraph 16).

16) Recycler emptying

Automatic Cash starts the JCM iPRO recycler emptying routine.

The parameters are the following:

{ "tipo": 51, "opName": USERNAME, "full": 0/1 }

The "**full**" field is optional and indicates whether the emptying of the recycler must be total or partial (emptying to petty cash level).
The possible values are as follows:

1. full = 0 Total emptying
2. full = 1 emptying to petty cash level

If the field is omitted, the default value is 0-total emptying.

The field "**opName**" is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the emptying is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

Automatic Cash response only confirms the start of the emptying operation.

Automatic Cash response only confirms the start of the emptying operation.

{ "tipo": 51, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

Warning: do not send the Recycler complete emptying command while Automatic Cash is performing any other operation.

Warning: do not send other commands to Automatic Cash until the hopper emptying operation has been completed.
To check when the emptying is completed use the command Emptying polling (see paragraph 16).

17) Stacker cancellation

With this command the Automatic Cash resets to zero the content of the coins cashbox and/or the banknotes stacker.

Attention: this command must be used only after the operator has physically emptied the coins cashbox and/or the banknotes stacker.

The parameters are the following

{ "tipo": 52, "peripheral": 0/1/2, "opName": USERNAME }

The field "**opName**" is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the cancellation is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

The "**peripheral**" field specifies which stacker we want to reset:

- peripheral = 0: the coins cashbox content is resetted
- peripheral = 1: the banknotes stacker content is resetted
- peripheral = 2: both coins cashbox and banknotes stacker content are resetted

Automatic Cash response only reports the result of the operation.

{ "tipo": 52, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

18) Emptying polling

With this command the machine will return the status of the current emptying in progress (be it coins or banknotes).

The parameters are the following

{ "tipo": 53 }

Automatic Cash response only reports the status of the emptying:

{ "tipo": 53, "req_status": 1, "empty_status": 0/1}

The "**empty_status**" fields has the following values:

- ***empty_status* = *0***: emptying completed
- ***empty_status* = *1***: emptying still in progress

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

Warning: after launching an emptying operations, we suggest to poll every 3 or 5 seconds.
Only when a empty_status = 0 response is received, it's possible to proceed with other operations.

19) Coin dispenser emptying

Automatic Cash starts the single denomination hopper (also called coin dispenser) emptying routine.

The parameters are the following:

{ "tipo": 54, "opName": USERNAME, "num": 1/2 }

The "**num**" field indicates which hopper we want to empty.
The possible values are as follows:

1. num = 1 total emptying hopper 1
2. num = 2 total emptying hopper 2

The field "**opName**" is optional and identifies the operator that do the request to the system; This field if present does automatically the login when the emptying is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

Automatic Cash response only confirms the start of the emptying operation.

{ "tipo": 54, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

Warning: do not send the Coin dispenser emptying command while Automatic Cash is performing any other operation.

Warning: do not send other commands to Automatic Cash until the hopper emptying operation has been completed.
To check when the emptying is completed use the command Emptying polling (see paragraph 16).

20) Door opening

With the door opening command, Automatic Cash unlocks the electronic lock of the machine door.
Once the electronic lock has been unlocked, the operator can open the machine door using the proper key.

The parameters are the following:

{ "tipo": 55, "wait_timeout": 0, "opName": USERNAME }

The field "**opName**" is optional and identifies the operator that do the request to the system.
This field if present does automatically the login when the opening is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

The "wait_timeout" parameters specifies the number of minutes after which the lock will be unlocked:

- if ***wait_timeout = 0***, lock is immediately unlocked
- if ***wait_timeout = 0:XX*****,** lock will be unlocked after XX seconds
- if ***wait_timeout = N***, lock will be unlocked after N minutes

Allowed values for wait_timeout field are:
- for the minutes, in the range of **0 and 60** minutes
- for the seconds, in the range of **0:01 and 0:59** seconds

The optional "**port**" parameter allows you to specify which electronic lock to unlock, in models where there are two different doors:

- if **port = 1**, the lock of the main body of the machine is unlocked
- if **port = 2**, the lock of the additional module is unlocked

Warning: if there is only one door, it is sufficient not to insert the "**port**" parameter in the call, or to insert it with the value **port = 1**.

Automatic Cash response only confirms the start of the unlocking routine.

{ "tipo": 55, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

Warning: do not send the Open door command while Automatic Cash is performing any other operation.

21) Cash closing

With the cash closing command, Automatic Cash returns the total amount inserted by customers and the total amount paid out by the machine since last cash closing, and also the current content of the machine.

The parameters are the following:

{ "tipo": 60, "opName": USERNAME }

The field "**opName**" is optional and identifies the operator that do the request to the system;
This field if present does automatically the login when the closing is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

Automatic Cash response is the following:

{ "tipo": 60, "req_status": 1,
"date": 1499296632, "start_date": 1499242632,
"total_in": 75000, "total_out": 11050,
"total_payments": 63950,
"total_operator_in": 10000, "total_operator_out": 2000,
"total_content": 200000,
"petty_cash": "136050",
"cash_income": 63950
}

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.
Other fields are:

- **date**: current date of the cash closing, expressed in UNIX timestamp format in seconds.
- **start_date**: starting date of the current cash closing, expressed in UNIX timestamp format in seconds. This is the date of the the previous cash closing.
- **total_in**: total amount inserted by customers, expressed in cents (in the example, 750 €)
- **total_out**: total amount paid out as rest by Automatic Cash, expressed in cents (in the example, 110.50 €).
- **total_payments**: total amount of the payments sent to Automatic Cash, expressed in cents (in the example 639.50 €).
  Please note that in this case it's equal to the difference between total_in and total_out; however this rule is not valid if there have been partial payments.
- **total_operator_in**: total amount of automatic refill operations (both coins and notes) made by the operators, expressed in cents (in the example 100€).
- **total_operator_out**: total amount of the sum of both withdrawals operations and emptying operations made by the operators (in the example 20€).
  Attention: emptying operations are the cancellations of the content of coins cashbox and banknotes stacker.
- **total_content**: current money content of Automatic Cash (coins and banknotes in recycle, coins and banknotes in cashbox and stacker).

- **petty_cash**: petty cash value
  Warning: this value is NOT an integer, because it could be in quanto può essere preceded by the character "*" to identificate a petty cash that was not correctly restored by the operator. In this case the value refers to the machine content at the time of the previous cash closing done.
  In the example the value (1360,50 €) has not the asterisk, so it means that non ha l'asterisco, therefore it means that it is the result of a correct petty cashrestore procedure.
- **cash_income**: cash income value, expressed in cents, as the difference between current content and petty cash value

Attention: Automatic Cash does not automatically manage the cash closings.
If for example a cash closing request is wrongly sent three day after the previous one, Automatic Cash will answer with data calculated in these last three days.

22) Refund

With refund command, Automatic Cash dispenses the amount of a given payment already completed.
The payment to be refunded is identified by its unique transaction ID.

Parameters are the following:

{ "tipo": 65, "id": 1454496777001, "opName": USERNAME }

The field "**opName**" is optional and identifies the operator that do the request to the system;
This field if present does automatically the login when the refund is processed by the Automatic Cash, if the field is not present, the operation will be saved to the logged user or to the "auth_login"

Automatic Cash response confirms the start of the refund operation:

{ "tipo": 65, "req_status": 1 }

Value of **"req_status"** field is detailed in Table 1 in paragraph 1.

In case of negative response (req_status: 2), for this specific command exists the following error code:

- **mess: 165**, indicates the payment cannot be refunded or the the payment has already been totally refunded

  Attention: only payments labeled as "***completed***" or "***partial***" can be refunded.

  Attention: it's possible to perform a refund only if the machine is enabled to do refunds. For more details check paragraph 32 (software configuration).

  Attention: if a payment refund cannot be completed because there's not enough cash inside the machine, it will be saved the amount partially refunded.
  In a second time, it's possible to repeat the refund operation and the remaining amount will be dispensed.

23) **Refund polling**

With the refund polling command, it is possible to check the status of the refund delivery operation.

Parameters are the following:

{ “tipo”: 66, “id”: 1617088551004 }

where the "**id**" field is the unique identifier of the transaction for which the refund was requested.

The Automatic Cash response reports the type of request, the response of the request and information about the status of the refund:

{ “tipo”: 66, “req_status”: 1, “id”: “1617088551004”
“refund_status”: “completed”, “refunded”: 5000, “toRefund”: 5000
}

Value of “req_status” field is detailed in Table 1 in paragraph 1.

The status of the reimbursement operation is given by the following subfields:

“refund_status”: urrent status of the refund operation and can assume the values:
1. “**pending**”: refund in progress
2. “**completed**”: refund payment terminated

“toRefund”: refundable amount of the transaction expressed in cents
“refunded”: amount paid as refund expressed in cents

In the previous example, the refund payment is complete and the entire transaction amount (€ 50) has been correctly refunded to the customer.

24) Withdrawals list

With the withdrawals list command, Automatic Cash returns the full list of withdrawals performed in the selected time interval.

Parameters are the following:

{ “tipo”: 70, “start_date”: 1499266236, “end_date”: 1499266836,
“opName”: USERNAME }

where “**start_date**” and “**end_date**” are the two filtering dates expressed as UNIX timestamp in seconds.

The field “**opName**” is optional and identifies the operator that do the request to the system;

Automatic Cash response contains the request type, the request result and an array containing the selected withdrawals:

{ **“tipo”: 70, “req_status”: 1,**
“withdrawals”: [
{ “date”: 1499266246, “value”: 5000, “operator”: operatore1 },
{ “date”: 1499266346, “value”: 2000, “operator”: operatore2 },
…
]
}

For each withdrawal the following fields are reported:

- “**date**”: date of the withdrawal (UNIX timestamp in seconds)
- “**value**”: amount withdrawn, expressed in cents (in the first example 50.00 €)
- “**operator**”: operator that performed the withdrawal (in the first example “operatore1”) **Attention**: in case of automatic withdrawal done by webservice, operator field will contain the username logged in at the time of the withdrawal.
  If there’s no operator logged in, operator field will have value “auth_login”.

25) Payments list

With the withdrawals list command, Automatic Cash returns the full list of payments performed in the selected time interval.

Parameters are the following::

{ "tipo": 71, "start_date": 1499266236, "end_date": 1499266836,
"opName": USERNAME }

where "**start_date**" and "**end_date**" are the two filtering dates expressed as UNIX timestamp in seconds.

The field "**opName**" is optional and identifies the operator that do the request to the system;

Automatic Cash response contains the request type, the request result and an array containing the selected payments:

{ **"tipo": 71, "req_status": 1,**
"payments": [
{ "date": 1499266246, "id": 1454496777001, status: "completed", "amount": 4500, "inserted": 5000, "rest": 500, "refund": 0, "operator": operatore1 },

{ "date": 1499266446, "id": 1464496777002, status: "partial", "amount": 1000, "inserted": 900, "rest": 0, "refund": 0, "operator": operatore1,
"cancelled_by": operatore1 },
…
]
}

For each payment the following fields are reported:

- "**date**": date of the payment (UNIX timestamp in seconds)
- "**id**": unique identifier of the transaction
- Attention: if "id"=0 it means the payment has not been sent by the management system, but it's been made in Automatic Cash manual mode.
- "**status**": status of the payment; values are the same detailed in paragraph b)
- "**amount**": amount of the payment, expressed in cents
- "**inserted**": amount inserted in the Automatic Cash by the customer, expressed in cents
- "**rest**": rest automatically dispensed by Automatic Cash, expressed in cents
- "**refund**": refunded amount, expressed in cents
- "**operator**": operator that managed the payment, it's the username logged in at the time of the payment
- "**cancelled_by**": present only in case of a cancelled payment by an operator(status "*returned*", "*partial*" o "*deleted*", as in paragraph b), reports the username of the operator who made the cancellation

26) Openings list

With the openings list command, Automatic Cash returns the full list of openings/closings performed in the selected time interval.

Parameters are the following:

{ “tipo”: 72, “start_date”: 1499266236, “end_date”: 1499266836,
“opName”: USERNAME }

where “**start_date**” and “**end_date**” are the two filtering dates expressed as UNIX timestamp in seconds.

The field “**opName**” is optional and identifies the operator that do the request to the system;

Automatic Cash response contains the request type, the request result and an array containing the selected openings/contents:

```
{ “tipo”: 72, “req_status”: 1,
  “openings”: [
        { “date”:  1499266246, “type”: 0, “operator”: operatore1 },
        { “date”:  1499266346, “type”: 1, “operator”: operatore1 },
        { “date”:  1499266346, “type”: 2, “operator”: operatore1 },
        …
    ]
}
```

For each opening/closing the following fields are reported:

- “**date**”: date of the opening/closing (UNIX timestamp in seconds)
- “**type**”: type of operation:
  - **type = 0**: door physically opened
  - **type = 1**: door physically closed
  - **type = 2**: opening authorization sent by the management system
- “**operator**”: operator that performed the opening/closing
  Attention: operator field contains the username logged in at the time of the operation.
  If there’s no operator logged in, operator field will have values “**unknown**” for the physical open/close operation, “**auth_login**” for the authorization operation.

27) Cash closing list

With the openings list command, Automatic Cash returns the full list of the cash closing performed in the selected time interval.

The parameters are the following:

{ "tipo": 73, "start_date": 1499266236, "end_date": 1499266836,
"opName": USERNAME }

where "**start_date**" and "**end_date**" are the two filtering dates expressed as UNIX timestamp in seconds.

The field "**opName**" is optional and identifies the operator that do the request to the system;

Automatic Cash response contains the request type, the request result and an array containing the selected cash closings:

```
{ "tipo": 73, "req_status": 1,
  "closings": [
        { "operator":  operatore1,
          "date": 1499296632, "start_date": 1499242632,
          "total_in": 75000, "total_out": 11050,
          "total_payments":  63950,
          "total_operator_in": 10000, "total_operator_out": 2000,
          "total_content": 200000,
          "petty_cash": "136050",
          "cash_income": 63950
        }

        { "operator":  operatore2,
          "date": 1499298761, "start_date": 1499296632,
          "total_in": 1000, "total_out": 900,
          "total_payments":  100,
          "total_operator_in": 0, "total_operator_out": 0,
          "total_content": 201000,
          "petty_cash": "200000",
          "cash_income":  100
       }
        …

    ]
}
```

For each cash closing, the name of the operator who has completed the operation is specified in the "operator" field with the same data returned when the cash closing is done.
For more details and the meaning of the various fields, refer to paragraph **21**).

28) Menu access

With the menu access command, the Automatic Cash open the configuration menu.
Depending on the user level, the menu will have different options; for details, refer to the Automatic Cash manual.

Parameters are the following:

{ “tipo”: 80, “userLevel”: 0/1/2,
“opName”: USERNAME, “password”: PASSWORD }

The "**userLevel**" field specifies the access level to the configuration menu:

- **userLevel = 0:** basic operator level.
  In this case the "**opName**" field is optional and identifies the operator who made the request on the client system.
  If present, the operation to access the configuration menu will be memorized as performed by this operator; if not present it will be stored as done by the user currently "**logged in**" or by the "auth_login" standard user.
  The "**password**" field is not used.

- **userLevel = 1:** operator level refill
  In this case, the "**opName**" field and the "**password**" field must correspond to a "**refill**" type user existing on the Automatic Cash.
  If the user does not exist, an error message with code 108 will be returned (see appendix I).
  If the user exists, that user will be automatically logged in.

- **userLevel = 2:** administrator level
  In this case, the "**opName**" field and the "**password**" field must correspond to an "**admin**" type user existing on the Automatic Cash.
  If the user does not exist, an error message with code 108 will be returned (see appendix I).
  If the user exists, that user will be automatically logged in

Automatic Cash response is the following:

{ **“tipo”: 80, “req_status”: 1 }**

Warning: after accessing the operator menu and completing the desired operations, **it is necessary to log out to exit the configuration menu**.

29) Reboot/Shutdown

With the reboot / shutdown command, the software can be restarted or the Automatic Cash can be switched off remotely.

{ “tipo”: 81, “opName”: USERNAME, “restart”: 0/1 }

The field “**opName**” is optional and identifies the operator that do the request to the system;

The "**restart**" field indicates whether to restart or shut down the machine:

- **restart: 1** reboots the software
- **restart: 0** turns off the machine completely

Automatic Cash response confirms the start of the reboot/shutdown operation:

{ “tipo”: 81, “req_status”: 1 }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

30) Version

With the version command, you can request the software version of Automatic Cash.

{ “tipo”: 82, “opName”: USERNAME }

The field “**opName**” is optional and identifies the operator that do the request to the system.

The Automatic Cash response simply reports the software version number in the "**version**" field.

{ “tipo”: 82, “req_status”: 1, “version”: “155” }

Value of **“req_status”** field is detailed in Table 1 in paragraph 1.

31) Peripherals configuration

With this command it's possible to change the coin/notes peripherals configuration by remote.

Regarding the banknotes recycler, it's possible to change the recycle denomination for each cassette and the maximum and petty cash levels.

Warning: if the Jcm recycler cassettes configuration has been changed, it is mandatory to manually replace the plastic spacers inside the recycler.
It is therefore always necessary a human operation to completed the configuration change.

Regarding the hopper, it's possible to change, for each denomination supported, the maximum and petty cash levels.

{ "tipo": 90, "opName": USERNAME, "config": CONFIG }

The field "**opName**" is optional and identifies the operator that do the request to the system.

The field "**config**" is a dictionary that contains the configuration details.
Supported configurations are the following:

- to configure the recycler cassettes, use tags "cassette_denom_1" and "cassette_denom_2" with the value of the requested denomination expressed in cents
- to configure the petty cash levels use tags "**cassette_min_level_1**" and "**cassette_min_level_2**"
- to configure the maximum notes levels use tags "**cassette_max_level_1**" and "**cassette_max_level_2**"
- to configure the coins maximum levels, for each denomination of coin supported by the hopper, use: "**hopper_max_level_YY**" where **YY** is the requested denomination expressed in cents.
  Ex: **hopper_max_level_50** refers to 50 cents, **hopper_max_level_100** to 1€
- to configure the coins minimum levels, for each denomination of coin supported by the hopper, use: "**hopper_min_level_YY**" where **YY** is the requested denomination expressed in cents.
  Ex: **hopper_min_level_5** refers to 5 cents, **hopper_min_level_200** to 2€
- to configure the acceptance for a specific coin, for each denomination of coin supported by the hopper use: "**hopper_accept_YY**" = **X**, where **YY** is the requested denomination expressed in cents.
  - value **X=1** indicates that the denomination of coin will be accepted
  - value **X=0** indicates that the denomination of coin will NOT be accepted

  Ex: **hopper_accept_5 = 0** indicates to NOT accept 5 cent coins
- to configure the payment for a specific coin, for each denomination of coin supported by the hopper use: **"hopper_recycle_YY" = X**, where **YY** is the requested denomination expressed in cents.
  - value **X=1** indicates that the denomination of coin will be used in payment
  - value **X=0** indicates that the denomination of coin will NOT be used in payment

  Ex: **hopper_recycle_5 = 0** indicates to NOT use 5 cent coins in payment

Attention: in case of recycler that supports 4 recycle denominations, it will be used the additional tags "**cassette_denom_X**", "**cassette_min_level_X**" e "**cassette_max_level_X**", where X takes the value "3" or "4".

Example (recycler with 2 cassettes):

"config":
{"cassette_denom_1": 500, # denomination expressed in cents of the first cassette (ex: 5€)
"cassette_denom_2": 2000, # denomination expressed in cents of the second cassette (ex: 20€)
"cassette_min_level_1": 20, # petty cash level for the first cassette (ex: 20 notes)
"cassette_max_level_1": 100, # maximum level for the first cassette (ex: 100 notes)
"cassette_min_level_2": 10, # petty cash level for the first cassette (ex: 10 notes)
"**cassette_max_level_2": 80,** # maximum level for the first cassette (ex: 80 notes)
"hopper_max_level_100": 100, # maximum level for 1€ coins set to 100 pieces
"hopper_min_level_20": 300, # petty cash level for 20 cents coins set to 300 pieces
"hopper_accept_5": 1, # 5 cent coins are accepted during counting
"hopper_recycle_5": 1, # 5 cent coins are used during payment
"hopper_accept_50": 0, # 50 cent coins are NOT accepted during counting
"hopper_recycle_50": 0, # 50 cent coins are NOT used during payment
}

Warning: all "config" fields are optional.
If a field is not present, no change related to that field will be performed on the machine.

Warning: if the value inserted for a single field is NOT allowed (for example a recycler cassette set to value 100), no change related to that field will be performed on the machine.

Warning: before changing the recycler cassettes configuration, it is mandatory to **completely empty** the recycler using the proper command "Recycler emptying".

Warning: maximum value allowed for the recycler cassettes maximum limit is "100" in case of JCM iPRO-RC recycler, "30" in case of JCM UBA-QUAD recycler and "60" in case of MEI recycler.

Warning: in case of MEI MDR recycler, the value of **"cassette_min_level_1"** refers to banknote "**banconota_1**" as returned by the call "**Machine Status**".
The same principle is applied to the other banknotes.

If the machine has single denomination hoppers (coin dispenser), it's possible to change the single hopper configuration, only if the hoppers are empty.

To change the hopper denomination, use the following tags:

"config":
{"coinDisp_denom_1": 50, # hopper 1 denomination in cent (in the example 50 cent)
"coinDisp_denom_2": 20, # hopper 2 denomination in cent (in the example 20 cent)
}

The Automatic Cash response confirms that the changes have been made.

{ "tipo": 90, "req_status": 1 }

Value of "req_status" field is detailed in Table 1 in paragraph 1.

32) Software configuration

With this command it's possible to change the Automatic Cash operational settings by remote.

{ "tipo": 91, "opName": USERNAME, "config": CONFIG }

The field "**opName**" is optional and identifies the operator that do the request to the system.

The field "**config**" is a dictionary that contains the configuration details.
Supported configurations are the following:

- **cash_max_limit**: single payment maximum value the machine can accept (ex: 1000, if the maximum limit is 1000 €)
- **single_payment: 0/1/2**
  If set to 0, the machine accepts a new payment request even if there is already a payment operation in progress; the new request will be queued and served as soon as possible.
  If set to 1, if there is already a payment in progress the new request will be rejected with error 105 (see Appendix I)
  If set to 2, if there is already an operation in progress each new request will be rejected with error 110 (see Appendix I)
- **allow_refund: 0/1/2**
  If set to 0, machine **DOES NOT** allow refunds.
  If set to 1, machine **ALWAYS** allows refunds.
  If set to 2, machine allows the complete refund for the completed payments; for the partially accepted payments the machine only refunds the amount inserted by the customer.
- **disabling_perc: X** where X between **0** and **10**
  It's the percentage of disabling of the hopper coins (for more details, please check the machine technical manual).
  Allowed values are "0" (no percentage set) or a value between 1 (that means 1%) and 10 (that means 10%).
- **coins_max_number**: maximum number of coins that the machine can payout during a change operation (ex: 10, if the maximum number of coins payable in a change operation is 10, regardless of the denomination of coin).
  For more details, please check the machine technical manual.

Warning: all "config" fields are optional.
If a field is not present, no change related to that field will be performed on the machine.

Warning: if the value inserted for a single field is NOT allowed (for example a recycler cassette set to value 100), no change related to that field will be performed on the machine.

Warning: after each Software configuration change **it is necessary to reboot the machine**.

The Automatic Cash response confirms that the changes have been made.

{ "tipo": 91, "req_status": 1 }

Value of "req_status" field is detailed in Table 1 in paragraph 1.

33) Petty cash setting

With this command is possible to change by remote the settings of the petty cash by value of Automatic Cash.

{ "tipo": 92, "opName": USERNAME, "pettycash": 50000 }

The field "**opName**" is optional and identifies the operator that do the request to the system.

The field "**pettycash**" is the value of the petty cash that we want to set; this value is expressed in cents, but the decimal part (last to digits) must have value 0.
Example: **"pettycash" : 50000** to set a petty cash of **500,00**.

Warning: if it's inserted a petty cash with decimal part different from 0, it will be automatically rounded down (example: 50012 will be rounded to 50000)

The Automatic Cash response contiains the current status of the machine after the petty cash setting.
This response has **"tipo" : 92** and the same fields of the response of the Status command detailed in paragraph 8.

{ "tipo": 92 *<see paragraph 8)>* **}**

It's very important to check the petty cash values that are automatically calculated:

- for evey coin denomination (i.e: subfield **moneta_100_min_level**)
- for evey banknote denomination (i.e: subfield **"min_level"** di **"banconota_1"**)

34) End petty cash restore

With this command is possible to communicate to Automatic Cash that has been completed the procedure to restore to petty cash level the machine content.

{ "tipo": 93 }

The Automatic Cash response just confirms the correct reception of the command:

{ "tipo": 93, "req_status": 1 }

Value of "req_status" field is detailed in Table 1 in paragraph 1.

The correct petty cash restore procedure is the following:

1. if not already set for the day, set the petty cash level for each denomination (command **Peripherals configuration, type 90**) or the petty cash value (command **Set petty cash, tipo 92**)
2. send the command to petty cash emptying for the coin peripheral (**Hopper emptying: type 50**) and the banknote peripheral (**Recycker emptying: type 51**)
3. physically remove the coins and notes from the cashbox and send the command to reset the cashbox content for coins and banknotes (**Stacker cancellation: type 52**)
4. send the command to start the refill (**Start refill: type 30**) to allow the operator to insert the missing coins and/or notes to reach the petty cash level set
5. send the command to end the refill (**End refill: type 31**)
6. send the command to signal the end of the petty cash restore procedure (**End petty cash restore: type 93**)

The current machine content will be used as the daily petty cash.

Appendix I

In case of response with "**req_status**": 2, the request cannot be satisfied because an error occurs. In this case there will be an additional parameter "**mess**" containing a code with the NACK reason.
All the available NACK messages and the possible solutions are detailed in the following table:

100	Generic error in the request package structure	Check if the package is correct
101	Unknown parameter "**tipo**" (type) in request package	Check if parameter "tipo" is correct
102	Generic error managing the request	Contact VNE
103	Error managing the payment or withdrawal request	Try again to send the request message; if error occurs again, contact VNE
104	Unknown parameter "**id**" in request package	Check that the unique identifier of the transaction is correct (it has to be equal to an ID returned by Automatic Cash in a previous request)
105	Cannot accept new payment; payment already in progress	Complete the pending payment (or modify option **single_payment=1**)
106	Cannot perform withdrawal: denomination passed as a parameter not present or not available	Check if the selected denomination is present and available on the machine
107	Cannot perform logout; payment still in progress	Complete the pending payment
108	Username and password passed as parameter do not correspond to a user existing on the machine	Check the correctness of the credentials passed as a parameter
110	Cannot process new request: there is an operation already in progress	Wait for the end of the operation in progress (or modify option **single_payment=2**)
113	Error during cancel request for a POS payment	Cannot cancel POS operation
114	Cannot cancel pending payment	User who send the cancel command is not the same user who requested
120	Authentication failed	Check validity of authentication string

Tabella 4: NACK message

Warning: in case of error 110, two additionals parameters will be included in the response message:

- **operation**: the type of the operation currently in progress:
    - 1: payment
    - 2: refund
    - 3: withdrawal
    - 4: refill
    - 5: emptying
- **user**: the user that has launched the operation currently in progress

Appendix II

Changelog:

version 2.0 :
- added the opName field in every commands to permit to login directly with only a command and have a faster response time.
- available from AC version **131**

version 2.2 :
- added new command List of cash closings
- added new command Access configuration menu
- added new commands Reboot/Shutdown
- added new command Machine Version
- added some parameters to different commands
- completely supported from AC version **172**

version 2.4 :
- added maximum and petty cash levels for recycler cassettes and hopper in Status (paragraph 8)
- added Change procedure description for the Start refill command (paragraph 9)
- added Emptying polling command (paragraph 16)
- added Peripherals configuration comand (paragraph 31)
- added Software configuration comand (paragraph 32)
- paragraphs indexed by number instead of letters
- available from AC version **248**

version 2.6 :
- added support for JCM reycler with 4 recycle denominations
- added banknote stacker content detail
- added support for manual refill
- available from AC version **252**

version 2.7:
- added new tags in Status command to indicate acceptance/recycling of each denomination of coins (paragraph 8)
- added new tags in Peripherals configuration comand to set acceptance/recycling of each denomination of coins (paragraph 31)
- available from AC version **297**

version 2.8:
- added coins withdrawal with single denomination
- added support for credit/debit card payment
- available from AC version **306**

version 2.9:
- added setting of maximum number of coins payable as change
- changed management of payment cancellation with option “return” (paragraph 3)
- available from AC version **313**

version 2.10:
- added 3 cassette dispenser support in Status command (paragraph 8)
- added total "totalInRecycle" for single peripheral in command Status (paragraph 8)
- added support for manual refill dispenser cassettes (paragraph 11)
- added support for double opening in command Door opening (paragraph 19)
- added new polling refill command (paragraph 12)
- added new refund polling command (paragraph 22)
- available from AC version **360**

version 2.11:
- added new option in Software configuration command (paragraph 31)
- added new message error 110 to manage single operation configuration
- added warning for MEI MDR minimum level setting
- available from AC version **394**

version 2.12:
- added new subfield to Payment status polling response (paragraph 2)
- added timer in seconds in Door opening command (paragraph 2)
- available from AC version **396**

version 3.01:
- added support for single denomination hoppers (coin dispensers)
- added new command Coin dispenser emptying (paragraph 19)
- added tags to commands Status (par 8), Manual Refill (par 11) and Peripherals configuration (par 31)
- added support for authentication key in Payment Request (par 1) (only for specific installations)
- available from AC version **400**

version 3.02:
- added new command Petty Cash setting (par 33)
- added tag “cancelled_by” in delete payment polling (par 2)
- changed “opName” tag behavior in Delete pending payment command (par 3) (only for specific installations)
- available from AC version **413**

version 3.03:
- added new tags in response for command Cash Closing (par 21) and Cash closings list (par 27)
- available from AC version **415**

version 3.04:
- added new command to signal the end of Petty cash restore operation (par 34)
- added tag for single coin denomination stacker detail in Status command (par8)
- available from AC version **418**